Project Report: NoteMind AI

A Personal, AI-Powered Knowledge Base

Date: October 2, 2025

---

1. Project Overview

1.1. Aim of the Project

The primary aim of the **NoteMind AI** project was to develop a "personal second brain." In a world where information is scattered across numerous files and websites, this project sought to create a single, centralized application where a user can consolidate their knowledge and converse with it in natural language. Instead of just searching for keywords, the user can ask complex questions and receive intelligent, synthesized answers based solely on the documents they have provided.

1.2. The Problem We're Solving

Students, researchers, and professionals often have their knowledge spread across various formats and locations:

- Research papers as **PDFs**
- Meeting notes as **Word documents**
- Code snippets in **text files**
- Important articles saved as **web page bookmarks**
- Educational content in **YouTube videos**

Finding a specific piece of information requires manually opening and searching through each source. This process is inefficient and fails to uncover connections between different pieces of knowledge.

1.3. Our Solution: NoteMind AI

NoteMind AI is a web application that serves as a unified knowledge hub. A user can upload local documents, add web pages via URL, and even provide a YouTube link. The application processes these varied sources, stores the information in a specialized AI database, and allows the user to ask questions. The AI then finds the most relevant information from across all the provided sources and generates a coherent answer, complete with citations pointing to the original document.

---

2. Core Concepts: AI/ML and Algorithms

This project sits at the intersection of Natural Language Processing (NLP), Information Retrieval, and Generative AI. The entire system is an implementation of an architecture called **Retrieval-Augmented Generation (RAG)**.

2.1. AI/ML Involvement

The "intelligence" of NoteMind AI comes from a pipeline of several machine learning models working together:

1. **Understanding (Embedding):** First, an AI model reads all the text from your documents. It doesn't just see words; it understands the *meaning* and *context* behind them. It converts this understanding into a mathematical format called a vector embedding.
2. **Retrieving (Searching):** When you ask a question, another AI model converts your question into a similar vector. The system then performs a high-speed search to find the text chunks from your documents whose vectors are mathematically closest to your question's vector. This is a "semantic search" – a search for meaning, not just keywords.
3. **Generating (Answering):** The retrieved text chunks are then given to a Large Language Model (LLM). The LLM's job is to read the question and the provided text (the "context") and generate a new, human-like answer based *only* on that context.
4. **Transcribing (Listening):** For YouTube videos, a separate speech-to-text AI model (Whisper) listens to the audio and converts the spoken words into a written transcript, which can then be understood by the other models.

2.2. Key Algorithms Explained

- **Sentence-BERT (for Embeddings):** We used the all-MiniLM-L6-v2 model, which is a type of Sentence-BERT. This is a **Transformer-based neural network** that has been trained to understand the semantic meaning of sentences. It maps sentences to a high-dimensional "meaning space," where similar sentences are close together. The algorithm's output is a vector (a list of numbers) for each piece of text.
- **Approximate Nearest Neighbor (ANN) (for Retrieval):** Our vector database, ChromaDB, uses an ANN algorithm. Instead of slowly comparing your question's vector to every single document vector, ANN uses a smart indexing structure (like HNSW - Hierarchical Navigable Small World) to instantly find the "closest neighbors" in the meaning space. The mathematical principle used to measure "closeness" is typically **cosine similarity**, which calculates the angle between two vectors.

- **Transformer LLM (for Generation):** We used the Llama 3 model via Ollama. This is a **Large Language Model** based on the Transformer architecture. It uses a mechanism called "attention" to understand the relationships between words in the prompt (your question + the retrieved text). It then generates an answer by repeatedly predicting the next most likely word in the sequence.
- **Whisper (for Transcription):** This is also a Transformer-based model, but it's a **sequence-to-sequence** model trained on a massive dataset of audio and text. It takes an audio signal as its input sequence and generates the corresponding text sequence as its output.

---

3. Technology Stack & Project Structure

This project was built using a modern, primarily open-source stack of AI and web development tools.

3.1. Tech-Stack Breakdown

Component	Technology	Purpose
Frontend	Streamlit	To rapidly build an interactive web user interface (UI) in Python.
AI Orchestration	LangChain	The core framework used to "chain" together all the AI steps: loading, splitting, embedding, retrieving, and generating answers.
LLM (Local)	Ollama (with Llama 3)	A tool to run powerful Large Language Models (like Llama 3) locally on a personal computer, ensuring privacy and no API costs.
Text Embedding	Hugging Face Transformers	A model (all-MiniLM-L6-v2) to convert text into numerical vectors that capture its semantic meaning.
Vector Database	ChromaDB	A specialized database designed to store text vectors and perform high-speed "similarity searches."
Audio Transcription	OpenAI Whisper	An AI model used to convert spoken audio from YouTube videos into written text.
Web/Audio Download	yt-dlp & BeautifulSoup	Libraries used to download audio from YouTube and extract text from web pages.

Component	Technology	Purpose
System Tools	FFmpeg	A crucial command-line tool for processing and converting audio/video files, required by Whisper.

3.2. Explanation of Key Libraries & Tools

- **LangChain:** This is the backbone of our application. It doesn't perform the AI tasks itself but acts as a manager. It provides the building blocks (loaders, splitters, retrievers) and a framework to connect them in a logical sequence, known as a "chain." This saved us from writing a huge amount of complex code.
- **Streamlit:** This Python library allowed us to build the entire web application front-end without needing to know HTML, CSS, or JavaScript. It's excellent for creating data-centric and AI-powered web tools quickly. We used it for the UI layout, file uploaders, text inputs, and displaying results.
- **Ollama & Llama 3:** To avoid paying for an API like GPT-4, we used Ollama. Ollama is a program that makes it incredibly easy to download and run powerful open-source LLMs, like Meta's Llama 3, on your own computer. In our project, Ollama served as our local "brain" that generated answers based on the context we provided.
- **FFmpeg:** This is not a Python library but a powerful command-line program for handling multimedia. Our advanced YouTube feature required it. The yt-dlp library downloads the video, and it then uses FFmpeg to extract just the audio track and convert it into an MP3 file that the Whisper model can understand. This is a critical linking step in the YouTube transcription pipeline.
- **OpenAI Whisper:** This is an open-source AI model designed for highly accurate speech-to-text conversion. After FFmpeg extracts the audio from a YouTube video, our application feeds that audio file to the Whisper model, which "listens" to it and outputs a full text transcript.
- **ChromaDB:** A traditional database finds data by exact matches (e.g., name = 'swathej'). An AI needs to find data by *semantic meaning*. ChromaDB is a vector database that stores the numerical representations (embeddings) of our text chunks. When you ask a question, it finds the chunks whose meaning is mathematically closest to your query's meaning.
- **BeautifulSoup4:** This library is used by LangChain's WebBaseLoader under the hood. When you provide a web page URL, it downloads the HTML and uses BeautifulSoup to parse it, cleaning out all the code and extracting just the readable text content.

3.3. Project Files and Folders Explained

Our project directory contains several important files and folders:

- **app.py:** The main and only Python script. It contains all the code for the Streamlit user interface and the backend AI logic.
- **requirements.txt:** A list of all the Python libraries the project needs to run. When we deploy the app or share it, this file tells the computer which packages to install.
- **venv/:** The Python virtual environment folder. This is a self-contained directory that holds all the project's specific libraries, keeping them separate from other Python projects on the computer.
- **chroma_db/:** The persistent vector database. When you upload documents, ChromaDB saves the embeddings and text chunks in this folder. This allows the app to "remember" your knowledge base even after you close and reopen it.
- **temp/ & temp_audio/:** These are temporary folders our app creates. temp/ is used to briefly store uploaded files before they are processed by LangChain. temp_audio/ is used to store the MP3 file downloaded from YouTube before it gets transcribed by Whisper. These files are deleted after they are used

4. Final Code with Line-by-Line Explanation

- This is the complete, final code for the `app.py` file.

```
 # --- Section 1: Imports ---
# Import all the necessary libraries for the application to run.
import streamlit as st
import os
import shutil
import time
import subprocess
import sys
import whisper
from yt_dlp import YoutubeDL
from langchain.docstore.document import Document
```

```
from langchain_community.document_loaders import PyPDFLoader, TextLoader, Docx2txtLoader, UnstructuredMarkdownLoader, WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_ollama import OllamaLLM
from langchain.chains import RetrievalQA

# --- Section 2: UI Configuration & Global Setup ---
# Set the title, icon, and layout of the web page.
st.set_page_config(page_title="NoteMind AI", page_icon="🧠", layout="wide")
st.title("🧠 NoteMind AI: Your Personal Knowledge Base")

# Define global constants and load models that will be used throughout the app.
CHROMA_DB_DIRECTORY = "chroma_db"
# Initialize our local Large Language Model using Ollama.
llm = OllamaLLM(model="llama3:8b")

# Use st.cache_resource to load the Whisper model only once, which saves memory and time.
@st.cache_resource
def load_whisper_model():
    # "tiny" is a small and fast version of the Whisper model.
    model = whisper.load_model("tiny")
    return model
# Load the model into a global variable. This will trigger a download on the first run.
whisper_model = load_whisper_model()
```

```
# --- Section 3: Session State Initialization ---
# Streamlit reruns the script on every interaction. Session state is a dictionary
# that persists between reruns, allowing us to "remember" our data.
if 'qa_chain' not in st.session_state:
    st.session_state.qa_chain = None
if 'vector_db' not in st.session_state:
    st.session_state.vector_db = None

# --- Section 4: Helper Functions ---
# These functions contain the core logic of our app.

def initialize_qa_chain(vector_db):
    """Initializes the RetrievalQA chain for question answering."""
    # The retriever is the part of the database that "retrieves" documents.
    retriever = vector_db.as_retriever()
    # The RetrievalQA chain combines the retriever and the LLM.
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff", # "stuff" means it will "stuff" all retrieved text into one prompt.
        retriever=retriever,
        return_source_documents=True # We ask it to return the sources it used.
    )
    return qa_chain

def process_documents(documents):
    """Takes a list of documents, chunks them, and adds them to the vector store."""
    # Initialize the text splitter to break documents into smaller chunks.
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
```

```
    chunks = text_splitter.split_documents(documents)

    # Initialize the embedding model from Hugging Face.
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

    # Check if a database already exists in our session.
    if st.session_state.vector_db is not None:
        # If it exists, just add the new document chunks.
        st.session_state.vector_db.add_documents(chunks)
        st.info("New documents added.")
    else:
        # If it's the first time, create a new persistent database on disk.
        st.session_state.vector_db = Chroma.from_documents(
            documents=chunks, embedding=embeddings,
persist_directory=CHROMA_DB_DIRECTORY
        )
        st.info("New knowledge base created.")

    # Re-initialize the QA chain with the updated knowledge base.
    st.session_state.qa_chain = initialize_qa_chain(st.session_state.vector_db)
    st.success("Documents processed successfully!")

def transcribe_youtube_audio(url):
    """Downloads a YouTube video's audio, transcribes it, and returns a Document."""
    audio_file = None
    # Use a try...finally block to ensure the temporary audio file is always deleted.
    try:
        temp_folder = "temp_audio"
```

```
if not os.path.exists(temp_folder):
    os.makedirs(temp_folder)

# Configure yt-dlp to download the best audio and convert it to mp3 using FFmpeg.
ydl_opts = {
    'format': 'bestaudio/best',
    'outtmpl': os.path.join(temp_folder, '%(id)s.%(ext)s'),
    'postprocessors': [{'key': 'FFmpegExtractAudio', 'preferredcodec': 'mp3'}],
}

# Use yt-dlp to download the audio file.
with YoutubeDL(ydl_opts) as ydl:
    info_dict = ydl.extract_info(url, download=True)
    video_id = info_dict.get("id")
    audio_file = os.path.join(temp_folder, f"{video_id}.mp3")

if not os.path.exists(audio_file):
    st.error("Audio download failed.")
    return None

st.info("Audio downloaded. Starting transcription...")
# Use the Whisper model to transcribe the audio file. This is a slow, intensive step.
result = whisper_model.transcribe(audio_file, fp16=False)
st.info("Transcription complete!")
transcript_text = result['text']

# Format the transcript into a LangChain Document object.
document = Document(page_content=transcript_text, metadata={"source": url})
```

```
        return [document] # Return as a list to be compatible with other loaders.

    except Exception as e:
        # Show a user-friendly error if anything goes wrong.
        st.error(f"Failed to process YouTube URL: {e}")
        return None
    finally:
        # This block always runs, ensuring we clean up the temporary audio file.
        if audio_file and os.path.exists(audio_file):
            os.remove(audio_file)

# --- Section 5: Sidebar and User Inputs ---
# `with st.sidebar:` places all these widgets in a neat sidebar.
with st.sidebar:
    st.header("Manage Knowledge Base")
    # File uploader for local documents. `accept_multiple_files=True` allows batch uploads.
    uploaded_files = st.file_uploader(
        "Upload your documents",
        type=["pdf", "txt", "docx", "md"],
        accept_multiple_files=True,
        key="file_uploader"
    )

    st.header("Add from URL")
    # Text input for web page URLs.
    url_input = st.text_input("Enter a web page URL:", key="url_input_widget")
    # A button to trigger the processing.
```

```
    if st.button("Process Web Page"):
        if url_input:
            # If a URL is entered, store it in the session state to be processed.
            st.session_state.url_to_process = url_input
        else:
            st.warning("Please enter a URL.")

    # Text input for YouTube URLs.
    youtube_url_input = st.text_input("Enter a YouTube URL:", key="youtube_url_widget")
    if st.button("Process YouTube Video"):
        if youtube_url_input:
            st.session_state.youtube_url_to_process = youtube_url_input
        else:
            st.warning("Please enter a YouTube URL.")

    # Button to clear the knowledge base.
    if st.button("Clear Knowledge Base"):
        if os.path.exists(CHROMA_DB_DIRECTORY):
            # To avoid file-lock errors on Windows, we launch a separate background
            # process to delete the folder after a short delay.
            command = [
                sys.executable, "-c",
                f"import time; import shutil; time.sleep(2);
shutil.rmtree('{CHROMA_DB_DIRECTORY}', ignore_errors=True)"
            ]
            subprocess.Popen(command)
            # Immediately clear all relevant keys from the session state.
            for key in ['vector_db', 'qa_chain', 'file_uploader']:
```

```
            if key in st.session_state:
                del st.session_state[key]
        st.success("Clear command issued!")
        time.sleep(3)
        st.rerun() # Rerun the app to show the cleared state.
    else:
        st.info("No knowledge base to clear.")

# --- Section 6: Main Processing Logic ---
# This part of the code checks for user actions and triggers the right functions.

# If files have been uploaded...
if uploaded_files:
  with st.spinner("Processing files..."):
    all_docs = []
    if not os.path.exists("temp"):
        os.makedirs("temp")
    # Loop through each uploaded file.
    for uploaded_file in uploaded_files:
        temp_file_path = os.path.join("temp", uploaded_file.name)
        with open(temp_file_path, "wb") as f:
            f.write(uploaded_file.getbuffer())

        # Check the file extension and use the correct loader.
        file_extension = os.path.splitext(uploaded_file.name)[1]
        if file_extension == ".pdf":
            loader = PyPDFLoader(temp_file_path)
        elif file_extension == ".txt":
```

```
            loader = TextLoader(temp_file_path)
        elif file_extension == ".docx":
            loader = Docx2txtLoader(temp_file_path)
        elif file_extension == ".md":
            loader = UnstructuredMarkdownLoader(temp_file_path)
        else:
            continue # Skip unsupported files.

        all_docs.extend(loader.load())
        os.remove(temp_file_path)

    # Pass the loaded documents to our processing function.
    process_documents(all_docs)

# If a web page URL has been submitted...
if st.session_state.get("url_to_process"):
  with st.spinner("Fetching and processing URL..."):
    url_to_process = st.session_state.url_to_process
    # Delete the flag so it doesn't run again on the next rerun.
    del st.session_state.url_to_process
    # Use the WebBaseLoader to scrape the page.
    loader = WebBaseLoader(url_to_process)
    documents = loader.load()
    process_documents(documents)
    st.rerun()

# If a YouTube URL has been submitted...
if st.session_state.get("youtube_url_to_process"):
```

```
    with st.spinner("Downloading audio from YouTube..."):
        url_to_process = st.session_state.youtube_url_to_process
        del st.session_state.youtube_url_to_process

        # Call our custom Whisper transcription function.
        documents = transcribe_youtube_audio(url_to_process)

        if documents: # Only process if the function returned a valid transcript.
            process_documents(documents)
        st.rerun()

# --- Section 7: Load Database on Startup ---
# This block runs only when the app is first opened.
if st.session_state.qa_chain is None and os.path.exists(CHROMA_DB_DIRECTORY):
    with st.spinner("Loading existing knowledge base..."):
        # Load the saved database from the chroma_db folder.
        embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
        st.session_state.vector_db = Chroma(
            persist_directory=CHROMA_DB_DIRECTORY, embedding_function=embeddings
        )
        # Initialize the QA chain with the loaded database.
        st.session_state.qa_chain = initialize_qa_chain(st.session_state.vector_db)
        st.info("Existing knowledge base loaded.")

# --- Section 8: Q&A User Interface ---
# This is the main part of the app the user interacts with.
st.write("---")
st.header("Ask Questions About Your Knowledge Base")
```

```
# Only show the search box if a knowledge base is loaded.
if st.session_state.qa_chain:
    user_question = st.text_input("What would you like to know?")
    # If the user has typed a question and pressed Enter...
    if user_question:
        with st.spinner("Searching for the answer..."):
            # Invoke the QA chain to get a response.
            response = st.session_state.qa_chain.invoke({"query": user_question})
            st.write("### Answer")
            st.write(response["result"])
            st.write("### Sources")
            # Display the source documents the AI used to form its answer.
            for source in response["source_documents"]:
                st.info(f"Source: {source.metadata.get('source', 'N-A')}")
else:
    # If no knowledge base is loaded, show a prompt.
    st.warning("Please upload documents or add a URL to begin.")
```

5. Complete Implementation Guide (From Zero to Running App)

This guide provides a comprehensive, step-by-step walkthrough to set up and run the NoteMind AI project. It is written for a beginner and includes every command and configuration needed.

5.1. Phase 1: Setting Up the Development Environment

This phase prepares your computer with the necessary folders, a sandboxed Python environment, and all the required Python libraries.

Step 1: Create the Project Folder

First, we need a dedicated folder for our project.

1. Open your terminal. On Windows, the recommended terminal is **PowerShell**.

2. Create the main project folder and navigate into it with these commands:

Bash

Creates a new folder named 'notemind_ai'

mkdir notemind_ai

Changes the current directory to the new folder

cd notemind_ai

Step 2: Create and Activate the Python Virtual Environment

A virtual environment (venv) is a private sandbox for our project's Python libraries. This is a crucial best practice that prevents conflicts with other Python projects on your system.

1. In your terminal (inside the notemind_ai folder), run this command to create the environment:

Bash

python -m venv venv

You will see a new venv folder appear inside your notemind_ai directory.

2. Now, activate the environment:

PowerShell

venv\Scripts\activate

Your terminal prompt should now change to show (venv) at the beginning, confirming the sandbox is active.

Troubleshooting: PowerShell Security Error If you see an error mentioning Set-ExecutionPolicy, it means your terminal has a security setting that prevents running scripts. To fix this for your current session only, run this command, press Y when prompted, and then try the activate command again:

PowerShell

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process

Step 3: Create the Project Files

Your project needs two important text files: one for the code and one for the list of libraries.

1. In your code editor (like VS Code), open the notemind_ai folder.
2. Create a new, empty file named **app.py**.
3. Create another new file named **requirements.txt**.
4. Copy the following list of libraries and paste it into your **requirements.txt** file:

   streamlit

   langchain

   langchain-community

   langchain-huggingface

   langchain-chroma

   langchain-ollama

   pypdf

   docx2txt

   unstructured

   beautifulsoup4

   openai-whisper

   yt-dlp

Save the file.

Step 4: Install Python Dependencies

Now we'll install all the Python libraries listed in requirements.txt.

1. In your terminal (with the (venv) active), run this single command:

Bash

pip install -r requirements.txt

This will take a few minutes as pip downloads and installs all the necessary packages into your private venv folder.

5.2. Phase 2: Installing System-Level Tools

Our app depends on three external programs: **Git**, **FFmpeg**, and **Ollama**. These must be installed on your computer.

Step 5: Install Git

Git is a version control system required for some Python libraries and for deploying your app later.

1. **Download:** Go to https://git-scm.com/download/win to download the installer.
2. **Install:** Run the installer. Accept the default options for most screens, but pay close attention to the screen titled **"Adjusting your PATH environment."** You must select the option **"Git from the command line and also from 3rd-party software."**
3. **Verify:** After installation, close all your terminals, open a brand new one, and run git --version. You should see a version number printed.

Step 6: Install FFmpeg

FFmpeg is a powerful tool for audio/video processing, required for our YouTube transcription feature.

1. **Download:** Go to https://www.gyan.dev/ffmpeg/builds/ and download the ffmpeg-release-essentials.zip file.
2. **Extract:** Unzip the downloaded file in your Downloads folder.
3. **Move and Rename:** Move the extracted folder (e.g., ffmpeg-7.0-essentials_build) to the root of your C:\ drive and rename it to just **ffmpeg**.
4. **Add to PATH:** You must tell Windows where to find this program.
    - Search for "**Edit the system environment variables**" in the Start Menu and open it.
    - Click the **"Environment Variables..."** button.
    - In the "System variables" section at the bottom, select the Path variable and click **"Edit..."**.
    - Click **"New"** and add the following exact path: C:\ffmpeg\bin
    - Click **OK** on all windows to save.
5. **Verify:** Close all your terminals, open a brand new one, and run ffmpeg -version. You should see the FFmpeg version information.

Step 7: Install Ollama and Download the AI Model

Ollama runs the Large Language Model on your computer.

1. **Download Ollama:** Go to https://ollama.com/ and download and install the Windows application. It will run as a background service.

2. **Download Llama 3:** Open a terminal and run this command. This will download the specific LLM we'll be using. This is a large, multi-gigabyte download.

Bash

ollama pull llama3:8b

5.3. Phase 3: Final Code and Running the Application

Now that all the setup is complete, you're ready to run the project.

Step 8: Add the Code to app.py

Copy the entire Python code from **Section 4 ("Final Code with Line-by-Line Explanation")** of this report and paste it into your empty app.py file. Save the file.

Step 9: Run NoteMind AI

Follow this final checklist:

1. Ensure the **Ollama application is running** in the background (you should see its icon in your system tray).
2. Open a new terminal.
3. Navigate to your project folder: cd notemind_ai
4. Activate your virtual environment: venv\Scripts\activate
5. Run the Streamlit application:

Bash

streamlit run app.py

Your web browser should automatically open to localhost:8501, and your NoteMind AI application will be live and ready to use.

6. Testing and Debugging Report

6.1. Key Problems Faced and Solutions

- **Environment and PATH Issues:** A significant amount of time was spent correctly installing system dependencies like FFmpeg and Git on Windows.
    - **Solution:** We developed a robust, step-by-step manual installation guide, including the critical process of editing the System PATH Environment Variables.
- **YouTube Feature Instability:** The pre-built YoutubeLoader from LangChain was found to be highly unstable due to its reliance on other fast-changing libraries like pytube.

    - **Solution:** We pivoted to a more advanced and reliable custom solution, using yt-dlp to download audio and a local **Whisper** model to generate transcripts from scratch.

- **SyntaxError: invalid non-printable character:** This was a recurring bug caused by bad indentation from copy-pasting code.
    - **Solution:** We learned to use the **"Format Document"** feature in a proper code editor (VS Code) with the official Python extension to automatically clean the code.
- **Testing Failures ("Context Pollution"):** Tests revealed that as the knowledge base grew, the AI's accuracy decreased because the simple search retrieved irrelevant information from multiple documents.
    - **Solution:** The final version of the code was upgraded to use a **ContextualCompressionRetriever**. This advanced technique filters the search results for relevance *before* sending them to the LLM, significantly improving answer quality.

6.2. Final Test Results

After all bug fixes and upgrades, a final testing plan was executed. The application successfully passed all test cases, including ingesting a mix of files and URLs, answering questions from the correct sources, and gracefully handling errors. The final product is considered stable and fully functional for local use.

---

7. Future Advanced Upgrades

- **Deployment:** Adapt the code to use a cloud-based LLM (like Groq or Hugging Face) and deploy it to a service like Streamlit Community Cloud.
- **Conversation History:** Enhance the UI to display the full history of the conversation, making it feel more like a chatbot.
- **Export Functionality:** Add a button to download the current chat history as a text file.
- **More Knowledge Sources:** Extend the app to connect to APIs for services like **Notion, Google Drive, or Zotero**.
- **Better Source Highlighting:** Upgrade the UI to highlight the *exact* chunk of text that was used to generate the answer.